

TP04 noté : Le Jeu du Snake

Adrien ESCOURROU

January 15, 2026

Date limite de rendu : 23/01/2026 à 23h59

INTRODUCTION

Dans ce TP04, nous posons les fondations d'un jeu Snake fonctionnel et structuré. L'objectif est d'apprendre à modéliser un jeu en Orienté Objet en Python avec Pygame, en créant des acteurs simples mais clairement séparés: Snake (le serpent), Food (la nourriture) et Game (la logique générale du jeu). À la fin du TP, vous aurez une version opérationnelle capable d'afficher le serpent et la nourriture, et d'effectuer un déplacement basique sans gestion des collisions complexes.

Informations

Tout au long de ce TP vous allez utiliser une librairie graphique utile pour les jeux vidéos, elle se nomme *pygame*. Il vous faut d'abord l'installer :

```
1 pip install pygame
```

Dans la suite du TP vous allez utiliser des fonctionnalités de pygame telles que :

```
1 pygame.time # Pour la gestion du temps
2 pygame.event # Pour la gestion des evenements
3 pygame.draw # Pour dessiner des objets et/ou dessins
4 pygame.key # Pour la gestion des touches
5 pygame.display # Pour l'affichage
```

Cette liste est incomplète et la suite des fonctions peut se trouver ici : <https://www.pygame.org/docs/>

Critères de notation

Ce TP sera noté, il est donc primordial de comprendre le barème :

- Toutes les fonctions demandées devront être implémentées. Le nom des fonctions, variables et squelettes de code doit être respecté.
- L'utilisation de l'IA est autorisé, cependant cela doit respecté trois critères importants :
 - Le code fait ou amélioré grâce à l'IA doit être encadré comme ci-dessous :

```
1 ##### CODE IA #####
2 VOTRE CODE
3 #####
```

- Tout code IA non mentionné mais qui sera détecté amènera automatiquement à un 0.
- Tout projet qui comporte peu ou pas d'IA sera bonifié et valorisé en terme de note.
- Les outils graphiques présentés sont rudimentaires, toute amélioration graphique sera bonifié et valorisé.
- Un jeu non fonctionnel, c'est à dire qui ne comporte pas les fonctionnalités de base, sera sanctionné.
- Tout code ne respectant pas les consignes vus en cours, sur les pratiques de bon code ou bien les énoncés sera sanctionné.
- Tout jeu présentant des améliorations, autres que celles demandées, sera bonifié et valorisé.
- La bonne structure ou rédaction d'un fichier README sera valorisé ou bien sanctionné.

Ce TP sera coefficient 1, à titre de comparaison votre projet sera coefficient 3 et votre partiel coefficient 2. Par conséquent il représente tout de même 16,5% de la note de votre module.

*Tous vos résultats devront être publiés sur votre répertoire GIT partagé avec votre professeur en TP01. Ici vous nommerez un répertoire **TP04** et publierez votre code sous le nom **snake.py**.*

Objectifs

En terme d'objectifs pour ce TP04, vous allez devoir :

- Créer les classes Snake, Food, Game & MovingEntity.
- Utiliser des constructeurs.
- Afficher un serpent + une nourriture + déplacement basique.
- Gérer de l'encapsulation : contrôler l'accès à l'état interne (`_body`, `_x`, `_y`, etc.).
- Gérer du polymorphisme : traiter des objets différents uniformément via `update()` et `draw()`.
- Introduire des contrats (pré/post conditions) et limiter les effets de bord.
- Utiliser l'héritage pour factoriser (éviter duplication de logique de déplacement).
- Introduire des attributs de classe (paramètres globaux cohérents : taille de case, vitesse).
- Introduire des mutateurs robustes (validation, cohérence globale).
- Ajouter des règles : murs, collision avec soi, game over.

Spécifications fonctionnelles

Le jeu doit :

1. Ouvrir une fenêtre.
2. Afficher :
 - Un serpent
 - Une nourriture (une case)
3. Déplacer le serpent avec les flèches.
4. Déplacer continuellement le serpent à vitesse constante.

Pour cela, voici les consignes à respecter :

- Lire le clavier pour changer la direction
- Affichage d'un message "GAME OVER"
- Déplacer la tête du serpent continuellement avec changement de position
- Quand la tête atteint la nourriture :
 - La nourriture se repositionne
 - Le serpent grandit
 - Le score augmente

Contraintes techniques

En terme de code :

- Le serpent est une liste de *positions* (x, y) .
- La tête est toujours *body[0]*.
- $_dx, _dy$ valent $\pm CELL$ sur un axe et 0 sur l'autre.
- Ajout d'un paramètre global *CELL_SIZE* qui permet de gérer la grille.
- Ajout d'un paramètre global *DEFAULT_SPEED* qui permet de gérer la vitesse.
- Utilisation d'une liste *entities* et pouvant être appelée comme ci-dessous :

```
1 for e in entities: e.update(game)
2 for e in entities: e.draw(screen)
```

- En terme d'encapsulation :
 - Snake expose au moins une fonction (lecture)
 - Food expose *pos()* (lecture)
 - L'état interne est protégé (préfixe `_` + méthodes d'accès)

Étapes guidées

Étape A — Classes MovingEntity & Entity

Ces classes vont permettre de factoriser certaines fonctions afin de les utiliser en terme d'héritage. Ces classes seront composées :

- D'une fonction *set_cell_size(cls, value)* qui définit la taille de la cellule.
- D'une fonction *set_default_speed(cls, value)* qui définit la vitesse par défaut du serpent.
- D'une fonction *set_direction(dx, dy)* qui met à jour la direction.

L'initialisation doit avoir à minima :

```
1 class Entity:
2     def update(self, game):
3         pass
4
5     def draw(self, screen):
6         pass
7
8 class MovingEntity(Entity):
9     CELL_SIZE = 20
10    DEFAULT_SPEED = 10
11    def __init__(self):
12        self._dx = self.CELL_SIZE
13        self._dy = 0
```

Étape B — Classe Snake

Cette classe va concerner principalement tout ce qui appartient au serpent, à savoir la direction, les collisions etc ... Cette classe sera composée :

- De la gestion du game over :
 - Dans le cas des murs : si $x < 0$ ou $x \geq \text{WIDTH}$ (pareil pour y) $\rightarrow$ `game_over=True`
 - Dans le cas de la tête dans le reste du corps $\rightarrow$ game over
 - Arrêt du update quand il se produit un game over

- D'une fonction *update()* qui insère une nouvelle tête et renvoie la position de la tête
- D'une fonction *draw(screen)* qui dessine chaque segment du serpent
- D'une fonction *grow(n)* qui sert à augmenter la taille du serpent via la variable *_grow_pending*

L'initialisation de la classe doit avoir à minima :

```

1 class Snake(MovingEntity):
2     def __init__(self, x, y):
3         super().__init__()
4         self._body = [(x, y)]
5         self._grow_pending = 0

```

Étape C — Classe Food

Cette classe va concerner principalement tout ce qui appartient à la nourriture. Cette classe sera composée :

- D'une fonction *__init__()* qui choisit une position aléatoire alignée sur la grille
- D'une fonction *draw(screen)* qui dessine la nourriture
- D'une fonction *pos()* qui renvoie les positions (x, y) de la nourriture
- D'une fonction *respawn()* qui repositionne la nourriture
- Utilise *MovingEntity.CELL_SIZE* pour réapparaître sur la grille

Étape D — Classe Game

Cette classe va concerner principalement tout ce qui appartient au jeu et ses fonctionnalités. Cette classe sera composée :

- D'une initialisation à pygame
- Des appels aux classes Snake & Food
- D'une fonction *handle_events()* qui lit des événements au clavier

- D'une fonction *update()* qui met à jour les déplacements du serpent via les *entities*
- D'une fonction *draw()* qui met à jour les éléments graphiques
- D'une fonction *run()* qui va servir de boucle principale

L'initialisation de la classe doit avoir à minima :

```

1 class Game:
2     def __init__(self):
3         pygame.init()
4         ....
5         self.score = 0
6         ....
7         self.food = Food()
8         self.entities = [self.food, self.snake]
9         ....

```

Création d'un README

En tout premier, merci de créer un fichier README à la racine de votre projet où vous noterez les réponses aux questions posées. Un README est un fichier texte, mais pour les plus habitués vous pouvez l'écrire en Markdown (README.md) ce qui est plus sympa et majoritairement utilisé. (Cela sera valorisé niveau note). De plus, toutes les améliorations effectuées en dehors du TP doivent être notées dans ce README, la structure est libre (mais toute structure mal rédigée ou en dehors des standards sera sanctionné). Ce document devra noter rigoureusement comment lancer votre jeu, les commandes possibles et le but final du jeu. Il devra aussi noter les déplacements interdits ou les éventuelles limitations s'il y en a.

Méthode de rendu

Une fois le TP fini, il faudra notifier votre professeur par courriel à **adrien.escourrou.prof@gmail.com** avec les informations suivantes :

- Prénom et Nom des personnes du groupe
- Le lien complet vers le projet sur Git (ou éventuellement un lien vers un exécutable)

- Les éventuelles questions, limitations ou annexes que vous jugerez nécessaire.

Tout projet rendu après la date limite entrainera d’office un 0.

Questions

Merci de noter toutes les réponses aux questions dans le README.

1. Quels sont les rôles respectifs de Snake, Food, Game ?
2. Donnez un exemple concret où l’accès direct à `snake._body` pourrait casser le jeu.
3. Expliquez le polymorphisme ici : quelle “interface” implicite partagent Food/Snake ?
4. Pourquoi `Food.update()` existe alors qu’il ne fait rien ?
5. Quels éléments de logique sont factorisés grâce à `MovingEntity` ?
6. Pourquoi `CELL_SIZE` et `DEFAULT_SPEED` sont des attributs de classe et pas d’instance ?
7. Donnez un exemple où `set_cell_size()` protège le programme (valeurs invalides).
8. Aurait-on pu éviter l’héritage ici ? avantages/inconvénients.

Extensions (bonus de note)

Pour les plus rapides vous pouvez essayer de :

- Création d’un exécutable (afin de lancer le jeu facilement).
- Empêcher le serpent de faire demi-tour instantanément.
- Ajouter une grille visuelle (lignes fines).
- Ajouter un mode “pause”.
- Empêcher la nourriture de respawn sur le serpent (contrainte + boucle de tirage).
- Ajouter “pas de demi-tour” (contrainte sur `set_direction`).

- Ajouter un système d'états (menu, running, paused, game_over) via un pattern State.
- Ajouter des obstacles (classe Wall), toujours via entities.
- Ajouter une IA simple (snake autonome) pour illustrer polymorphisme/stratégie.